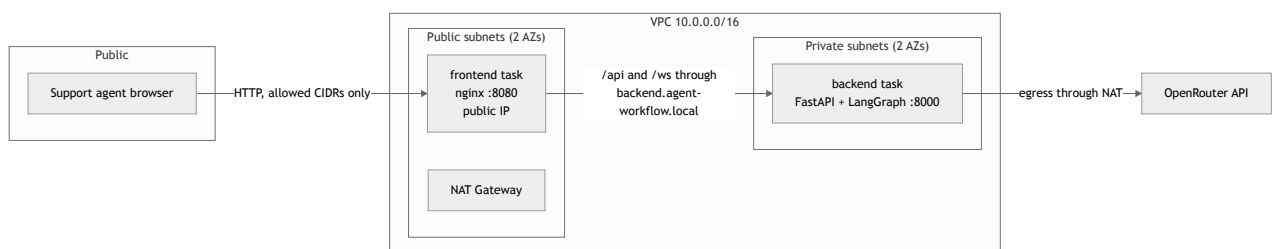


Design Document

1. [Architecture](#)
2. [LangGraph workflow](#)
3. [Security](#)
4. [CI/CD and operations](#)
5. [Tradeoffs and limits](#)

1. Architecture



Frontend and Backend are two independent ECS services

The frontend is nginx with the React application in a public subnet

The backend is FastAPI with the LangGraph workflow in a private subnet

The frontend proxies `/api` and `/ws` to the backend

Network

Item	Choice
Ingress	Frontend public IP, port 8080, <code>allowed_ingress_cidrs</code> only
Frontend to backend	Cloud Map private DNS, <code>backend.agent-workflow.local</code>
Backend	Private subnets, port 8000 from the frontend security group only
Egress	One NAT gateway for ECR, CloudWatch, and OpenRouter
TLS	HTTP only

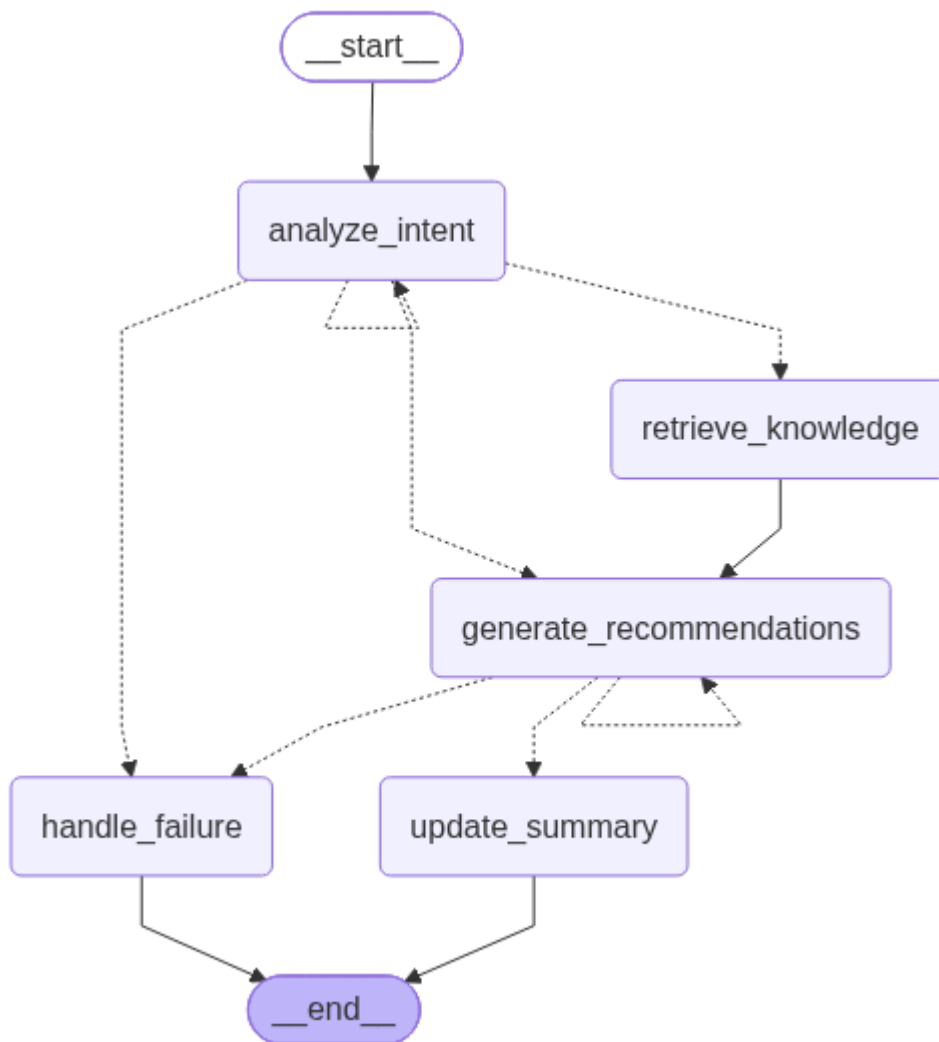
Security group chain: `allowed_ingress_cidrs` -> `frontend-sg:8080` -> `backend-sg:8000`. Access to the backend can only be reached via the security group of frontend

Observability

- Persist logs in AWS Cloudwatch under these namespace - `/ecs/agent-workflow/frontend` and `/ecs/agent-workflow/backend`

- Each run logs its start and end with the `conversation_id` and a short `run_id` . A node failure logs a traceback that can be analysed on CloudWatch namespace
- Both the Frontend and Backend logs can be inspected from the running container itself - No need for external logging services
-

2. LangGraph workflow



The design that I have chosen is a simple stateful workflow that first analyses the intent from the message and decides which node to move on to.

This is done via the conditional edge that routes the decisions to either the retrieval, recommendation, failure node and the node itself if no definitive routing can be made.

Each node is checkpointed in the memory and can be accessed by the `thread_id`

Node	Type	Function
<code>analyze_intent</code>	LLM	Intent classification

Node	Type	Function
retrieve_knowledge	deterministic	Keyword search over documents
generate_recommendations	LLM	Draft reply, necessary information, next action
update_summary	LLM	Short summary of the conversation
handle_failure	deterministic	Reduced sidebar when a node fails

Memory

- `messages` uses an append reducer that the checkpointer keeps, so each run sees the full conversation.
- The last 20 messages go without change. The function `update_summary` adds the summary into the previous conversations
- At the moment there is no Long Term Memory as it might be a scope creep in the assessment

3. Security

- **Network.** The backend service has no public IP and no internet route. Only the frontend security group reaches port 8000.
- **IAM.** Each service has its own execution role. Only the backend service is able to read the secrets from the Secrets Manager
- **Secrets.** The API keys are managed in Secrets Manager and set in terraform state during the build
- **Documents** The knowledge base documents are stored in the container image. This is developed such for this interview assessment and simplicity sake
- **Logging** Logs are routes to each CloudWatch group for each service

4. CI/CD and operations

Workflow	Trigger	Steps
ci.yml	pull request, push to <code>main</code> or <code>uat</code>	ruff, pytest, frontend lint and build, Terraform <code>fmt -check</code> and <code>validate</code>
deploy.yml	push to <code>main</code>	Build and push both images tagged with the Git SHA, <code>terraform apply</code> with that tag, wait for <code>ecs services-stable</code> , output the frontend URL

First installation:

1. `terraform apply` in `terraform/bootstrap` makes the bucket, the CI role, and the ECR repositories

2. In GitHub, set the secrets `AWS_DEPLOY_ROLE_ARN` and `TF_STATE_BUCKET`, the variable `AWS_REGION`, and the `production` environment.
3. Put the API key in Secrets Manager with `aws secretsmanager put-secret-value`.
4. Push to `main`. The pipeline builds the images, pushes them, and applies the root stack.

Run the `application_url_command` output to get the console address to view the Support Agent.
`terraform destroy` removes everything.

5. Tradeoffs and limits

1. Conversation state is in memory. The LangGraph checkpointer is stored in memory. Ideally you would want to have both short term (LangGraph checkpointer) and long term memory.

For Production - Short term memory are informations that is stored in the context window of the LLM which is useful per session. Long term memory are informations that persist across different sessions and typically stored on S3 or DynamoDB

2. No Load Balancer on ingress. The frontend runs as one task with a public IP on HTTP, because the assignment has no domain and no certificate.

For Production, put an ALB in front of the frontend with an ACM certificate and an HTTPS listener